# 4-2 指令的寻址方式

## 1. 指令寻址

- 基本概念
  - **指令寻址**：寻找下一条将要执行的指令地址。
  - **PC 作用**：程序计数器 PC 的内容指示待执行指令的地址。
  - **顺序执行**：取出当前指令后，PC 自动增加，形成下一条指令地址。
- 顺序寻址
  - **含义**：程序按主存中的指令顺序依次执行。
  - **适用场景**：普通非转移类指令。
- 跳转寻址
  - **跳转指令**：执行到转移类指令时，下一条指令地址不再简单来自顺序自增。
  - **记忆点**：相对寻址通常使用“当前指令下一条指令地址”作为基准。

## 2. 数据寻址

- 基本概念
  - **数据寻址**：确定本条指令所需操作数的位置。
  - **寻址特征字段**：由于数据寻址方式很多，指令中通常设置字段指出采用哪种寻址方式。
  - **形式地址 A**：指令地址字段中给出的地址信息，通常不一定是真实地址。
  - **有效地址 EA**：操作数所在的真实地址。
- 一地址指令格式
  - **操作码 OP**：说明指令要执行什么操作。
  - **寻址特征**：说明本条指令采用哪种寻址方式。
  - **形式地址 A**：配合寻址方式计算或得到操作数。

## 3. 常见数据寻址方式

- 立即寻址
  - **放的是什么**：指令字中直接放操作数本身。
  - **形式地址 A**：不是操作数地址，而是操作数本身，也称立即数。
  - **数据形式**：数据通常以补码形式存放。
  - **优点**：取出指令后即可获得操作数，不需要为取操作数再访问存储器。
  - **缺点**：A 的位数限制了立即数的表示范围。
  - **速记**：直接放数。
- 直接寻址
  - **放的是什么**：指令字中直接放操作数的真实地址。
  - **公式**：`EA = A`。
  - **访问过程**：根据 A 访问主存，取出操作数。
  - **优点**：寻找操作数简单，不需要专门计算地址；执行阶段对主存只访问一次。
  - **缺点**：A 的位数限制操作数寻址范围；若要改变操作数位置，必须修改指令中的 A。
  - **速记**：直接放地址。
- 隐含寻址
  - **放的是什么**：指令字中不明显给出操作数地址。
  - **地址来源**：操作数地址隐含在操作码或某个寄存器中。
  - **典型例子**：另一个操作数隐含在 ACC 中。
  - **优点**：指令字中少一个地址字段，可缩短指令字长。
  - **缺点**：需要增加保存操作数或隐含地址的硬件。
  - **速记**：藏一个地址。
- 间接寻址
  - **放的是什么**：指令字中的 A 给出“操作数有效地址所在地址”。
  - **一次间接**：A 是地址的地址，先访问 A 所指单元，得到 EA，再访问 EA 取操作数。
  - **二次间接**：A 是地址的地址的地址，需要多次访存才能得到目标操作数。
  - **优点**：可扩大寻址范围，因为有效地址 EA 的位数可大于形式地址 A 的位数。
  - **缺点**：执行阶段需要多次访存。
  - **速记**：放地址的地址。
- 寄存器寻址
  - **放的是什么**：地址码字段给出寄存器编号。
  - **公式**：`EA = R`，R 表示寄存器编号。
  - **操作数位置**：操作数存放在寄存器中。
  - **特点**：寄存器不是主存的一部分，与主存地址空间无关。
  - **优点**：访问速度快，通常不需要访问主存取操作数。
  - **限制**：寄存器数量有限，地址字段只能表示有限个寄存器。
  - **速记**：指出寄存器，数放寄存器。
- 寄存器间接寻址
  - **放的是什么**：指令字中给出寄存器编号。
  - **寄存器内容**：寄存器中保存操作数所在主存单元的地址。
  - **公式**：`EA = (R1)`，其中 `(R1)` 表示取出寄存器 Ri 中的值。
  - **访问过程**：先读寄存器得到主存地址，再访问主存取得操作数。
  - **特点**：综合了寄存器寻址和间接寻址的特点。
  - **速记**：指出寄存器，地址放寄存器。
- 基址寻址
  - **放的是什么**：指令给出形式地址 A，基址寄存器 BR 给出基准地址。
  - **公式**：`EA = A + (BR)`。
  - **基址寄存器 BR**：由操作系统或管理程序根据主存空间情况确定。
  - **面向对象**：基址寻址面向操作系统。
  - **作用**：操作系统可通过给 BR 赋值，把用户程序安置在主存的某一空间区域。
  - **特点**：程序执行过程中 BR 的值通常不可由用户随意改变。
  - **速记**：基址寄存器，面向 OS。
- 变址寻址
  - **放的是什么**：指令给出形式地址 A，变址寄存器 IX 给出偏移变化量。
  - **公式**：`EA = A + (IX)`。
  - **与基址寻址相似点**：都是寄存器内容与形式地址相加得到有效地址。
  - **关键区别**：变址寄存器内容由用户设定，程序执行过程中其值可变；指令字中的 A 通常不变。
  - **主要用途**：处理数组和循环程序。
  - **数组理解**：可设 A 为数组首地址，不断改变 IX 的内容，形成数组中任一元素地址。
  - **速记**：变址寄存器，面向用户。
- 相对寻址
  - **放的是什么**：指令中的 A 给出相对位移量。
  - **公式**：`EA = (PC) + A`。
  - **PC 含义**：这里的 PC 是当前指令下一条指令的地址。
  - **常见用途**：转移类指令。
  - **相对位移量**：转移目标地址与当前基准地址之间的距离，由形式地址 A 给出。
  - **正负表示**：A 可正可负，常用补码表示。
  - **速记**：PC 的值 + 给出的 A = 目的地址。

## 4. 对比速记

- 按“指令里放了什么”区分
  - **立即寻址**：直接放数。
  - **直接寻址**：直接放地址。
  - **隐含寻址**：地址藏在操作码或寄存器中。
  - **间接寻址**：放地址的地址。
  - **寄存器寻址**：放寄存器编号，操作数在寄存器。
  - **寄存器间接寻址**：放寄存器编号，操作数地址在寄存器。
  - **基址寻址**：A 加 BR，BR 面向操作系统。
  - **变址寻址**：A 加 IX，IX 面向用户。
  - **相对寻址**：PC 加 A，常用于转移。
- 按 EA 公式区分
  - **立即寻址**：A 即操作数，不计算 EA。
  - **直接寻址**：`EA = A`。
  - **间接寻址**：`EA = (A)`，多级间接可继续取地址。
  - **寄存器寻址**：操作数在寄存器，地址码给寄存器编号。
  - **寄存器间接寻址**：`EA = (R1)`。
  - **基址寻址**：`EA = A + (BR)`。
  - **变址寻址**：`EA = A + (IX)`。
  - **相对寻址**：`EA = (PC) + A`。
- 容易混淆点
  - **基址 vs 变址**：二者公式相似，但 BR 面向操作系统，IX 面向用户。
  - **直接 vs 立即**：直接寻址的 A 是地址，立即寻址的 A 是操作数。
  - **寄存器 vs 寄存器间接**：寄存器寻址中操作数在寄存器；寄存器间接寻址中主存地址在寄存器。
  - **相对寻址的 PC**：通常指当前指令取出后、已经自增到下一条指令地址的 PC。

by joshuji 由 幕布 发布